

```
!pip install gensim
```

```
Collecting gensim
```

```
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
```

```
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
```

```
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
```

```
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
```

```
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
```

```
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
```

```
27.9/27.9 MB 48.2 MB/s eta 0:00:00
```

```
Installing collected packages: gensim
```

```
Successfully installed gensim-4.4.0
```

```
# =====  
# Import Required Libraries for the Lab  
# =====
```

```
# --- Word Embedding Utilities ---
```

```
import gensim
```

```
import gensim.downloader as gensim_api
```

```
from gensim.models import KeyedVectors as KV
```

```
# --- Numerical Computations ---
```

```
import numpy as np
```

```
# --- Plotting & Visualization ---
```

```
import matplotlib.pyplot as plt
```

```
# --- Dimensionality Reduction (t-SNE) ---
```

```
from sklearn.manifold import TSNE as TSNE_Reducer
```

```
# =====  
# Download & Load Pre-trained Embedding Models  
# =====
```

```
import gensim.downloader as gensim_data
```

```
# Load Google News Word2Vec (300 dimensions)
```

```
w2v_model = gensim_data.load("word2vec-google-news-300")
```

```
# Load GloVe Wikipedia Gigaword (100 dimensions)
```

```
glove_model = gensim_data.load("glove-wiki-gigaword-100")
```

```
print("Word2Vec model loaded successfully!")
```

```
print("GloVe model loaded successfully!")
```

```
[=====] 100.0% 1662.8/1662.8MB downloaded
Word2Vec model loaded successfully!
GloVe model loaded successfully!
```

```
word_list = [

    # Politics
    "president", "minister", "senator", "governor", "diplomat",

    # Business
    "entrepreneur", "startup", "corporation", "profit", "economy",

    # Healthcare
    "physician", "pharmacist", "therapy", "diagnosis", "vaccine",

    # Birds
    "eagle", "sparrow", "pigeon", "parrot", "owl",

    # Sea Animals
    "shark", "whale", "dolphin", "octopus", "seal",

    # Digital Technology
    "database", "algorithm", "network", "cybersecurity", "processor",

    # Cities
    "tokyo", "berlin", "mumbai", "sydney", "toronto",

    # Natural Disasters
    "earthquake", "tsunami", "volcano", "flood", "hurricane"

]
```

```
# =====
# Extract Vectors from Word2Vec Model
# =====

valid_words = []
vector_data = []

for word in word_list:
    if word in w2v_model:
        valid_words.append(word)
        vector_data.append(w2v_model.get_vector(word))

# Convert into NumPy array
vectors_word2vec = np.array(vector_data)

# Print required output format
print("Total selected words:", len(valid_words))
```

```
print("Vector shape:", vectors_word2vec.shape)
```

```
# Print first word vector exactly like sample output
if len(vectors_word2vec) > 0:
    print(vectors_word2vec[0])
```

```
Total selected words: 40
```

```
Vector shape: (40, 300)
```

```
[-1.34887695e-02 -1.20117188e-01  1.44531250e-01  2.89306641e-02
-2.31933594e-02 -2.45117188e-01  9.66796875e-02 -1.54296875e-01
 2.53906250e-01 -1.80664062e-01 -1.08398438e-01  3.73535156e-02
-1.59179688e-01  5.52368164e-03  1.22558594e-01  2.15820312e-01
 6.25000000e-02  6.00585938e-02 -1.17675781e-01 -1.96289062e-01
-2.32421875e-01  1.81640625e-01  4.51660156e-02 -2.97851562e-02
 1.29882812e-01 -2.18750000e-01 -8.10546875e-02 -1.57226562e-01
-1.53320312e-01 -1.66015625e-01 -1.06445312e-01 -1.17797852e-02
-2.47070312e-01  1.29882812e-01  4.25781250e-01  1.61132812e-02
-4.07714844e-02  1.49414062e-01 -9.68933105e-04  1.46484375e-01
-6.64062500e-02 -1.05468750e-01  1.12304688e-01  1.78710938e-01
-2.02148438e-01 -2.34375000e-01 -1.33789062e-01  8.39843750e-02
-1.84570312e-01  2.51953125e-01 -2.25830078e-02  4.88281250e-02
-6.17675781e-02  2.74658203e-02  8.78906250e-02  1.23535156e-01
-1.80664062e-01  1.45263672e-02  6.05468750e-02 -1.07910156e-01
 8.10546875e-02  1.59179688e-01 -7.17773438e-02  2.94921875e-01
 3.32031250e-02  1.69921875e-01  1.54296875e-01  3.73535156e-02
 2.36328125e-01  1.49414062e-01  2.71484375e-01  4.78515625e-02
-1.81640625e-01 -2.14843750e-01 -7.71484375e-02 -5.46875000e-02
-1.24511719e-01  6.29882812e-02  3.30078125e-01 -6.05468750e-02
 1.90429688e-01  2.38281250e-01 -4.58984375e-02  1.79687500e-01
 2.08740234e-02  2.55126953e-02 -3.04687500e-01 -1.89453125e-01
-5.46875000e-02  1.36718750e-01  2.28515625e-01 -1.69921875e-01
 5.44433594e-02 -7.56835938e-02 -1.53320312e-01 -5.22460938e-02
-7.47070312e-02  1.33789062e-01  9.57031250e-02  7.71484375e-02
 1.10839844e-01 -2.23632812e-01  8.54492188e-02  9.17968750e-02
-2.92968750e-03  8.20312500e-02 -2.06298828e-02 -1.82617188e-01
-1.56250000e-01 -7.01904297e-03 -3.85742188e-02 -2.08007812e-01
 6.73828125e-02  8.39843750e-02  1.22558594e-01 -7.03125000e-02
 1.91406250e-01 -7.03125000e-02  3.55468750e-01  1.31835938e-01
-7.08007812e-02 -1.21093750e-01 -6.12792969e-02 -2.37304688e-01
-2.91748047e-02 -2.65625000e-01 -2.23632812e-01  3.56445312e-02
 3.14453125e-01  6.88476562e-02  5.39550781e-02 -1.02050781e-01
 1.02539062e-01  3.44238281e-02  1.11328125e-01  1.07910156e-01
 1.32812500e-01 -1.10351562e-01  1.17675781e-01  2.27539062e-01
-1.65039062e-01  3.90625000e-03 -7.37304688e-02 -1.13281250e-01
-1.64062500e-01  6.12792969e-02  2.60009766e-02  7.76367188e-02
 3.57421875e-01  1.13281250e-01 -1.16699219e-01  1.60156250e-01
-1.11328125e-01  1.39648438e-01 -1.88476562e-01  1.14257812e-01
-2.20703125e-01  3.68652344e-02  5.51757812e-02  4.00390625e-02
-1.20117188e-01 -1.25000000e-01  1.61132812e-01 -2.13623047e-02
 2.59765625e-01 -9.81445312e-02 -4.54101562e-02  1.26953125e-01
 4.33593750e-01  2.63671875e-01  3.51562500e-01  4.46777344e-02
 1.43554688e-01 -2.91015625e-01  1.43554688e-01 -1.07910156e-01
 2.81982422e-02  1.32446289e-02  5.22460938e-02 -4.02832031e-03]
```

```
-2.53906250e-01 -1.29882812e-01 1.85546875e-02 -1.59179688e-01
 2.30468750e-01 -5.34667969e-02 9.96093750e-02 7.20214844e-03
-2.99072266e-02 2.57812500e-01 5.98144531e-02 5.88378906e-02
-4.27246094e-02 3.59375000e-01 -1.09863281e-01 -4.02832031e-03
-6.59179688e-02 1.45507812e-01 1.84570312e-01 -1.15722656e-01
-1.72851562e-01 1.45507812e-01 2.38281250e-01 -6.83593750e-02
 5.93261719e-02 1.47705078e-02 2.21679688e-01 -2.70996094e-02
-1.14746094e-01 -1.27929688e-01 7.47070312e-02 3.56445312e-02
-5.29785156e-02 1.50390625e-01 2.23632812e-01 2.55859375e-01
-3.80859375e-02 -2.89062500e-01 5.98144531e-02 8.39843750e-02
```

```
# =====
# Extract Vectors from GloVe Model
# =====

# Filter words that exist in GloVe vocabulary
valid_words_glove = []
glove_matrix = []

for w in word_list:
    if w in glove_model.key_to_index:
        valid_words_glove.append(w)
        glove_matrix.append(glove_model.get_vector(w))

# Convert list to NumPy array
vectors_glove = np.asarray(glove_matrix)

# Print information in the same style
print("Total selected words:", len(valid_words_glove))
print("Vector shape:", vectors_glove.shape)

# Display first vector as sample
if vectors_glove.shape[0] > 0:
    print(vectors_glove[0])
```

```
Total selected words: 40
Vector shape: (40, 100)
[-0.064549 -0.13812  0.50017  0.41434  0.45832 -0.048331 -0.21651
 0.34987 -0.83235 -0.62282 -0.40099 -0.31978  0.42928 -0.026035
-0.16171 -0.32513  0.77604 -0.39852 -0.67013 -0.55876 -0.39863
-0.28143  0.68614  0.29229 -0.7146  0.36995 -0.45309 -0.23193
 0.76916 -0.031589 1.0603  1.5446 -0.13272  0.33701 -0.97561
 0.73617  0.60841  0.558 -0.39274  0.1121 -1.0474 -0.03692
 0.99908 -0.044925 -0.41186  0.19605  0.093281 -0.59623 -0.60765
-0.4742 -0.30162 -0.70128 -0.20055  0.99519  0.20088 -2.5636
-0.14627  0.77824  1.4913  0.29724 -0.12084 -0.060082  0.079881
-0.38348  0.41371 -0.41284  0.61702  1.1316 -0.047434  0.2848
 0.57694 -0.45501 -0.76359 -1.1659  0.068946 -0.22498  0.18691
 0.64142 -1.7505 -0.091248  0.85848 -0.52203 -0.38122 -0.43335
-0.73706 -0.51609 -0.13884  0.65128  0.66747 -2.1334  0.93429
 0.24036 -0.43398  0.86494 -0.78319 -0.032875 -0.19761 -0.23146
```

```
-0.20256  0.1193  ]
```

```
# =====
# Apply t-SNE Dimensionality Reduction
# =====

from sklearn.manifold import TSNE

# Assume vectors_word2vec is already available
# Example shape: (40, 300)

# Initialize t-SNE
tsne_reducer = TSNE(
    n_components=2,          # Reduce embeddings to 2 dimensions
    perplexity=10,
    learning_rate=200,
    random_state=42,
    init='random'           # Optional: better reproducibility
)

# Fit and transform vectors
tsne_result = tsne_reducer.fit_transform(vectors_word2vec)

# Print shapes for confirmation
print("Original Vectors Shape:", vectors_word2vec.shape)
print("t-SNE Reduced Shape:", tsne_result.shape)

# Display first 5 transformed coordinates
print("\nFirst 5 t-SNE coordinates:")
for i, coord in enumerate(tsne_result[:5]):
    print(f"{valid_words[i]}: {coord}")

# Optionally, show first 5 original vectors for comparison
print("\nFirst 5 Original Vectors:")
for i, vec in enumerate(vectors_word2vec[:5]):
    print(f"{valid_words[i]}: {vec[:5]} ...") # Show first 5 dimensions for brevity
```

```
Original Vectors Shape: (40, 300)
t-SNE Reduced Shape: (40, 2)
```

```
First 5 t-SNE coordinates:
president: [ 59.69107 -36.436157]
minister: [109.92719 -43.272343]
senator: [ 93.06012 -84.46398]
governor: [ 74.307785 -69.19527 ]
diplomat: [134.66045 -66.23612]
```

```
First 5 Original Vectors:
president: [-0.01348877 -0.12011719  0.14453125  0.02893066 -0.02319336] ...
minister: [-0.13671875 -0.01519775  0.06640625 -0.16601562 -0.10986328] ...
```

```
senator: [ 0.13085938 -0.02966309 -0.0378418  0.06176758 -0.26757812] ...
governor: [-0.16992188 -0.15234375 -0.21972656  0.01501465 -0.06591797] ...
diplomat: [ 0.24902344  0.14453125  0.12988281  0.30664062 -0.03833008] ...
```

```
# =====
# Visualize t-SNE Word Embeddings
# =====

import matplotlib.pyplot as plt

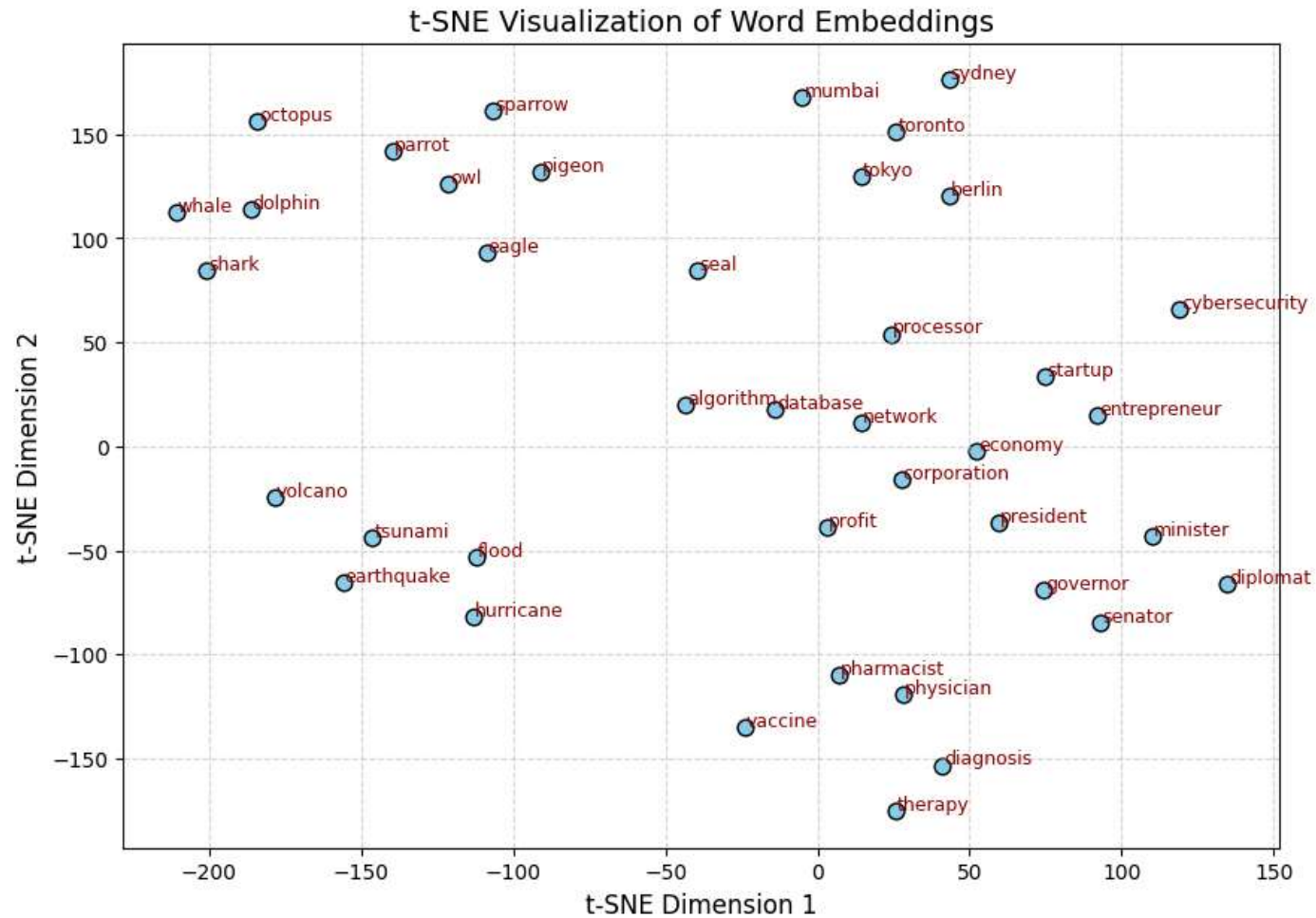
# Set figure size
plt.figure(figsize=(10, 7))

# Scatter plot of t-SNE coordinates
plt.scatter(tsne_result[:, 0], tsne_result[:, 1], color='skyblue', s=60, edgecolor='k')

# Annotate each point with corresponding word
for idx, word in enumerate(valid_words):
    plt.text(tsne_result[idx, 0] + 0.5, tsne_result[idx, 1] + 0.5, # slight offset
            word, fontsize=9, color='darkred')

# Title and labels
plt.title("t-SNE Visualization of Word Embeddings", fontsize=14)
plt.xlabel("t-SNE Dimension 1", fontsize=12)
plt.ylabel("t-SNE Dimension 2", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)

# Show plot
plt.show()
```



```
# =====
# Apply t-SNE on GloVe Word Vectors
# =====

from sklearn.manifold import TSNE

# Assume vectors_glove is already extracted (shape example: 40 x 100)

# Initialize t-SNE reducer
tsne_glove = TSNE(
    n_components=2,          # Reduce embeddings to 2 dimensions
    perplexity=10,
    learning_rate=200,
    random_state=42,
```

```

    init='random'          # Optional: improves reproducibility
)

# Fit and transform GloVe vectors
tsne_glove_result = tsne_glove.fit_transform(vectors_glove)

# Print shapes for verification
print("Original GloVe Vectors Shape:", vectors_glove.shape)
print("t-SNE Reduced Shape:", tsne_glove_result.shape)

# Display first 5 t-SNE coordinates with corresponding words
print("\nFirst 5 t-SNE Coordinates:")
for i, word in enumerate(valid_words_glove[:5]):
    print(f"{word}: {tsne_glove_result[i]}")

# Optionally, display first 5 original GloVe vectors (truncated for readability)
print("\nFirst 5 Original GloVe Vectors (first 5 dimensions):")
for i, vec in enumerate(vectors_glove[:5]):
    print(f"{valid_words_glove[i]}: {vec[:5]} ...")

```

```

Original GloVe Vectors Shape: (40, 100)
t-SNE Reduced Shape: (40, 2)

```

```

First 5 t-SNE Coordinates:

```

```

president: [-36.702618  45.309612]
minister:  [-42.477356  23.884722]
senator:    [-44.172863  70.31979 ]
governor:   [-28.887173  72.99385 ]
diplomat:   [-67.25165   39.551186]

```

```

First 5 Original GloVe Vectors (first 5 dimensions):

```

```

president: [-0.064549 -0.13812  0.50017  0.41434  0.45832 ] ...
minister:  [-1.5181  -0.74831  0.26892  0.63476  0.32357] ...
senator:   [-0.36424  0.62971  0.57543 -0.70793  0.74759] ...
governor:  [-0.21276 -0.57194  0.61748 -0.86632  0.91587] ...
diplomat:  [-0.41875 -0.3099   1.0402   0.34739  0.12454] ...

```

```

# =====
# Visualize t-SNE for GloVe Embeddings
# =====

```

```

import matplotlib.pyplot as plt

```

```

# Set figure size
plt.figure(figsize=(10, 7))

```

```

# Scatter plot with points styled
plt.scatter(tsne_glove_result[:, 0], tsne_glove_result[:, 1],
           color='lightgreen', s=70, edgecolor='black')

```

```

# Annotate each point with the corresponding word

```



```

# Annotate each point with the corresponding word
for idx, word in enumerate(valid_words_glove):
    plt.text(tsne_glove_result[idx, 0] + 0.3, # slight x-offset
             tsne_glove_result[idx, 1] + 0.3, # slight y-offset
             word, fontsize=9, color='darkblue')

# Add title and axis labels
plt.title("t-SNE Visualization of GloVe Word Embeddings", fontsize=14)
plt.xlabel("t-SNE Dimension 1", fontsize=12)
plt.ylabel("t-SNE Dimension 2", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)

# Show plot
plt.show()

```

